

Introducción a las Bases de Datos

1. ¿Qué es una Base de Datos?

1.1 Definición y concepto general

Una **base de datos** es un sistema que permite **almacenar, organizar, manipular y recuperar datos** de forma estructurada y eficiente.

En términos sencillos, se trata de una **colección organizada de información** que se guarda electrónicamente en un ordenador o servidor, y que puede ser **consultada por múltiples usuarios al mismo tiempo**.

Pero no basta con almacenar datos: lo importante es que se **puedan gestionar fácilmente** mediante un conjunto de herramientas o programas llamados **Sistemas de Gestión de Bases de Datos** (SGBD o DBMS, en inglés).

Definición técnica:

Una base de datos es un conjunto de datos relacionados, organizados de manera que permiten un acceso rápido, su gestión eficiente y la integridad de la información.

1.2 ¿Por qué son necesarias las bases de datos?

Imagina que un supermercado guarda los productos y ventas en hojas de papel o archivos Excel. ¿Qué pasaría cuando hay miles de productos, precios, proveedores y tickets? ¿Y si varios empleados quisieran consultar o modificar la misma hoja a la vez?

Las bases de datos permiten **centralizar toda la información**, controlar quién accede a ella y asegurar que siempre sea **consistente y actualizada**, incluso si hay muchos usuarios trabajando simultáneamente.



💬 1.3 Ejemplos cotidianos de uso

Las bases de datos están por todas partes. Algunos ejemplos reales:

Situación real	¿Qué información se almacena?
Banco	Datos de clientes, cuentas, tarjetas, movimientos, préstamos...
Plataforma de vídeo en streaming	Usuarios, series/películas, listas de favoritos, historial de visionado
Red social	Perfiles de usuarios, publicaciones, comentarios, reacciones, mensajes
Hospital	Pacientes, historiales médicos, citas, diagnósticos, medicación
Centro educativo	Estudiantes, cursos, calificaciones, horarios, asistencia
Tienda online	Productos, pedidos, métodos de pago, direcciones, valoraciones

Cada una de estas aplicaciones depende de **una o varias bases de datos para funcionar correctamente**.

🔍 1.4 Diferencia entre dato e información

Este es un concepto fundamental en informática:

Concepto	Definición	Ejemplo
Dato	Elemento básico de información, sin contexto	"Carlos", "500", "13/02/2024"
Información	Conjunto de datos organizados que tienen sentido	"Carlos compró un portátil de 500 € el 13/02/2024"

Los datos son materia prima. La información es el resultado de procesarlos.

Las bases de datos **almacenan datos**, pero el verdadero valor está en la capacidad de **procesarlos y convertirlos en información útil** para tomar decisiones.

1.5 Tipos de datos básicos en una base de datos

A la hora de diseñar una base de datos, es muy importante indicar **de qué tipo serán los datos** que se guardarán en cada columna de una tabla. Esto permite garantizar que se **almacenen correctamente** y que se puedan **realizar operaciones apropiadas** (como cálculos con números, búsquedas por texto, ordenación por fechas...).

Tipos de datos más comunes:

Tipo de dato	Ejemplo	Descripción
Enteros (INT)	1, 15, 230, -12	Números sin decimales (para cantidades, identificadores, unidades...)
Decimales (DECIMAL o FLOAT)	3.14, 1500.50	Números con parte decimal, útiles para precios, porcentajes, medidas...
Texto corto (CHAR, VARCHAR)	"Pepe", "Correo"	Cadenas de caracteres, para nombres, direcciones, códigos, etc.
Texto largo (TEXT)	"Este producto es excelente..."	Comentarios, descripciones largas, artículos...
Fecha y hora (DATE, DATETIME)	"2024-04-17"	Para almacenar fechas o fechas y horas con formato estándar
Booleano (BOOLEAN)	TRUE / FALSE (1 / 0)	Para almacenar respuestas lógicas o activaciones (sí/no, activo/inactivo...)

Importancia:

- Si defines bien el tipo de datos desde el principio, **mejoras la integridad y el rendimiento** del sistema.
- Si los defines mal, puedes provocar **errores** y pérdida de coherencia (ej: guardar un número de teléfono como INT y perder el 0 inicial).

2. ¿Por qué usamos Bases de Datos?

Este bloque tiene como objetivo que el alumnado comprenda **qué problemas resuelven las bases de datos** frente a los métodos tradicionales de almacenamiento de información, y **qué beneficios concretos aportan** a nivel técnico, organizativo y de seguridad.

2.1 Problemas del almacenamiento tradicional en archivos

Antes de que se usaran bases de datos modernas, las organizaciones almacenaban información en **documentos de texto, hojas de cálculo o incluso papel**. Este enfoque resultaba viable solo con volúmenes muy pequeños de datos, pero a medida que el sistema crece, aparecen numerosos problemas:

A) Redundancia de datos

Redundancia significa que la misma información aparece **varias veces en diferentes lugares**. Esto sucede, por ejemplo, cuando el nombre y la dirección de un cliente se guardan en múltiples archivos distintos (facturas, pedidos, incidencias...).

Problemas:

- Ocupa más espacio innecesariamente.
- Hace más difícil actualizar los datos: si cambias la dirección en un archivo, otros pueden quedar desactualizados.

B) Inconsistencia de los datos

La **inconsistencia** ocurre cuando hay **información contradictoria** en distintos archivos sobre una misma entidad.

Ejemplo:

En el archivo de facturación aparece que Pedro vive en “Calle Luna 12”, pero en el archivo de pedidos pone “Calle Sol 5”.

Consecuencias:

- Pérdida de fiabilidad de los datos.
- Dificultades para generar informes y tomar decisiones.

⊗ C) Dificultad de acceso concurrente

Cuando los datos están almacenados en archivos independientes, **no es fácil que varios usuarios accedan o editen los mismos datos a la vez.**

 **Ejemplo:** Dos empleados no pueden abrir al mismo tiempo una hoja Excel sin riesgo de sobrescribir información.

Problemas:

- Riesgo de errores por sobrescritura.
- Ineficiencia en entornos colaborativos.
- Imposibilidad de llevar un control de cambios en tiempo real.

D) Seguridad limitada

El almacenamiento tradicional ofrece **poco control sobre quién puede ver o modificar la información.** A menudo, cualquier persona con acceso al archivo puede leer o cambiar los datos.

Consecuencias:

- Posibles accesos no autorizados.
- Dificultad para aplicar niveles de acceso o roles.
- Peligro de pérdida o corrupción accidental.

🌸 2.2 Ventajas de las Bases de Datos

Frente a los problemas mencionados, los sistemas modernos de bases de datos permiten una **gestión eficiente, segura y estructurada de la información**. A continuación, se detallan sus principales ventajas:

☑ A) Integridad de los datos

Una base de datos permite definir reglas que **garantizan que los datos almacenados son correctos y coherentes**.

📌 Ejemplos de integridad:

- Que no se permita introducir una fecha inválida.
- Que no se guarden registros sin clave principal.
- Que una tabla no pueda referenciar a otra que no existe.

Esto reduce errores y mejora la confianza en los datos almacenados.

🔒 B) Seguridad y control de acceso

Los SGBD permiten definir **quién puede acceder, modificar o eliminar** información, y aplicar permisos por usuario, grupo o tipo de acción.

📌 Ejemplo:

- El departamento de recursos humanos puede acceder a los datos personales de los empleados, pero no puede modificar las nóminas.
- Un usuario “lectura” puede ver información pero no alterarla.

♻ C) Recuperación ante fallos

Los sistemas de bases de datos están diseñados para soportar mecanismos de:

- **Copia de seguridad (backups).**
- **Restauración de datos** tras un fallo del sistema.
- **Transacciones atómicas:** las operaciones se completan correctamente o no se aplican (principio de **todo o nada**).

Esto permite minimizar el impacto de errores, cortes de luz o fallos humanos.

D) Acceso concurrente controlado

Los SGBD permiten que **varios usuarios trabajen a la vez sobre los mismos datos sin generar conflictos**.

 ¿Cómo se consigue?

- Mediante sistemas de **bloqueo de registros** y control de transacciones.
- Cada cambio queda registrado y el sistema garantiza la coherencia de los datos.

Esto permite trabajar en equipo, generar informes en paralelo o procesar múltiples pedidos sin interferencias.

3. Componentes de un sistema de base de datos

Un sistema de base de datos no es solo un archivo con datos, sino una **arquitectura completa** que combina software, hardware, usuarios y normas de funcionamiento. Conocer sus componentes permite entender cómo se crean, gestionan y mantienen las bases de datos modernas.

¿Qué entendemos por "sistema de base de datos"?

Un **sistema de base de datos** es el conjunto de elementos que permiten **almacenar, gestionar y consultar datos de forma segura, eficiente y estructurada**.

Está formado por:

- Un software (SGBD)
- Los datos en sí
- Los usuarios que lo utilizan
- El hardware sobre el que se ejecuta

3.1 Sistema Gestor de Base de Datos (SGBD o DBMS)

Definición

El **Sistema Gestor de Base de Datos (SGBD)** es el **software principal** que permite:

- Definir la estructura de los datos (tablas, relaciones...)
- Almacenar y recuperar datos
- Aplicar restricciones e integridad
- Gestionar usuarios y permisos
- Proteger y recuperar la información

Funciones principales del SGBD

Función	Descripción breve
Creación de estructuras	Permite definir tablas, campos, relaciones, vistas, índices...
Manipulación de datos	Ejecuta consultas: INSERT, SELECT, UPDATE, DELETE
Control de concurrencia	Garantiza que varios usuarios accedan sin errores
Seguridad y control de acceso	Define quién puede ver, modificar o borrar información

Función	Descripción breve
Copias de seguridad y recuperación	Previene la pérdida de datos por fallos
Control de integridad	Asegura que los datos sean válidos y coherentes

Ejemplos de SGBD más utilizados:

- **Relacionales:**
 - MySQL / MariaDB
 - PostgreSQL
 - Oracle Database
 - Microsoft SQL Server
- **No relacionales (NoSQL):**
 - MongoDB
 - Redis
 - Cassandra

 *Todos estos SGBD permiten trabajar con miles o millones de datos en tiempo real, con integridad y seguridad.*

3.2 Tipos de usuarios en un sistema de base de datos

El uso de una base de datos no es igual para todos. Se definen **roles o perfiles** con diferentes responsabilidades y niveles de acceso:

Tipo de usuario	Funciones principales
Administrador de BD (DBA)	Instala, configura, optimiza y protege la base de datos
Diseñador de base de datos	Define el modelo de datos, tablas, relaciones y reglas de negocio
Programador	Desarrolla aplicaciones que interactúan con la base de datos
Usuario final	Usa formularios, consultas o programas para acceder a los datos

En un entorno real, **cada tipo de usuario tiene permisos distintos** para mantener la seguridad y la eficiencia del sistema.

3.3 Hardware necesario

Aunque muchas bases de datos se usan desde la nube hoy en día, es importante conocer el componente físico que soporta todo el sistema:

Elementos clave del hardware:

- **Servidor** o equipo donde se aloja la base de datos.
- **CPU**: Procesador potente para responder consultas rápidamente.
- **Memoria RAM**: Fundamental para gestionar operaciones concurrentes.
- **Almacenamiento (Disco duro o SSD)**: Donde se guardan los datos.
- **Red**: Para permitir el acceso remoto (intranet o Internet).

Niveles de complejidad

- Un estudiante puede tener **una base de datos en su portátil** (MySQL o SQLite).
- Una empresa puede tener **bases de datos distribuidas en varios servidores** (físicos o en la nube).
- Plataformas como Google, Netflix o Amazon usan **sistemas complejos con millones de transacciones por segundo**.

3.4 Software y aplicaciones relacionados

Además del SGBD principal, un sistema de base de datos incluye software adicional que interactúa con él:

Tipos de software vinculados:

Tipo	Ejemplo / Función
Interfaces gráficas (GUI)	MySQL Workbench, DBeaver, DB Browser for SQLite
Lenguajes de programación	Java, Python, PHP, C# (para conectar aplicaciones)
Aplicaciones cliente	ERP, CRM, gestores de stock o TPV
Herramientas de administración	Paneles web, dashboards, monitores de rendimiento

Herramientas de backup y auditoría Respaldos automáticos, registros de acceso

 *El ecosistema del software permite que la base de datos no sea un elemento aislado, sino parte activa de todo un sistema informático.*

Conclusión del punto 3

Un sistema de base de datos es mucho más que una tabla con datos. Implica:

- Un software central (SGBD)
- Un conjunto de usuarios con distintos roles
- Un soporte técnico (hardware)
- Aplicaciones o programas que permiten explotar la información

Comprender estos elementos es clave para diseñar, administrar y utilizar correctamente una base de datos profesional.

4. Tipos de Bases de Datos

Existen diferentes tipos de bases de datos según **cómo organizan, almacenan y relacionan la información**. No todas las bases de datos funcionan igual, y elegir el tipo adecuado depende del tipo de datos que se manejan, la estructura del sistema y los objetivos del proyecto.

En esta unidad veremos los principales tipos de bases de datos utilizados en la actualidad:

4.1 Bases de Datos Relacionales (RDBMS)

¿Qué son?

Son las bases de datos más utilizadas en entornos empresariales y académicos. Se basan en el **modelo relacional**, donde los datos se almacenan en **tablas** compuestas por filas y columnas, y las relaciones entre esas tablas se definen mediante **claves primarias y externas**.

Características:

- Usan el lenguaje **SQL** (Structured Query Language).
- Fuerte estructura y reglas (tipos de datos, relaciones...).
- Muy adecuadas para datos estructurados y relaciones complejas.

✓ Ventajas:

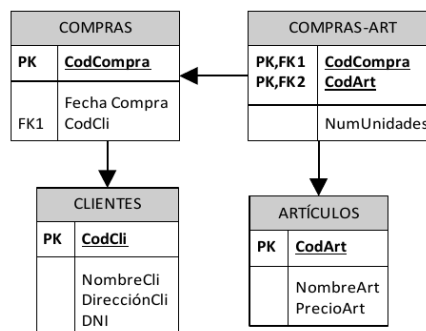
- Alta integridad y consistencia.
- Seguridad y control de acceso.
- Facilidad para hacer consultas complejas (JOIN, GROUP BY...).

⚠ Inconvenientes:

- Menos flexibles si los datos no tienen una estructura clara.
- Puede escalar peor con volúmenes masivos de datos heterogéneos.

📁 Ejemplos:

- MySQL
- PostgreSQL
- Oracle
- SQL Server
- SQLite



ORACLE
DATABASE

MySQL

PostgreSQL

Microsoft
SQL Server

4.2 Bases de Datos NoSQL (Not Only SQL)

¿Qué son?

Este tipo de bases de datos surgió como respuesta a las **limitaciones del modelo relacional** en sistemas que manejan datos no estructurados, muy variados o en tiempo real.

"NoSQL" no significa que no usen SQL, sino que no se basan exclusivamente en el modelo relacional.

Tipos de NoSQL:

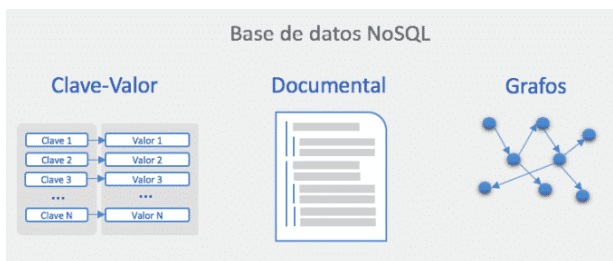
Tipo	¿Cómo almacenan los datos?	Ejemplo
Documentales	Como documentos JSON o BSON	MongoDB
Clave-valor	Como un diccionario o mapa	Redis
Columnar	Por columnas en vez de por filas	Cassandra, HBase
De grafos	Nodos y relaciones (como un mapa)	Neo4j

✓ Ventajas:

- Flexibles y escalables horizontalmente.
- Muy rápidas para lectura/escritura en grandes volúmenes.
- Ideales para datos no estructurados o semiestructurados.

⚠ Inconvenientes:

- Menor control de integridad (no garantizan relaciones).
- No son adecuadas para sistemas muy normativizados (como bancos).



4.3 Bases de Datos Orientadas a Objetos

¿Qué son?

Este tipo de bases de datos almacena los datos como objetos, igual que en la programación orientada a objetos (POO). Un objeto incluye atributos y métodos, y puede estar relacionado con otros objetos.

Ejemplo:

Un objeto Empleado puede tener como atributos el nombre, DNI y salario, y como métodos calcularSueldo().

✓ Ventajas:

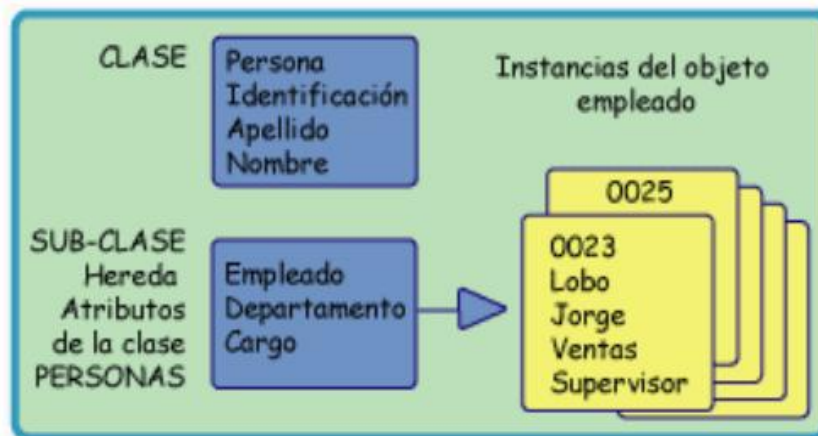
- Muy útiles cuando ya se trabaja con lenguajes como Java o C#.
- No es necesario convertir objetos en filas/columnas.

⚠ Inconvenientes:

- Menos conocidas y usadas.
- No tan eficientes para consultas complejas como las relacionales.

📁 Ejemplos:

- db4o
- ObjectDB



🔗 4.4 Bases de Datos Jerárquicas

🔗 ¿Qué son?

Este modelo almacena los datos en una estructura **tipo árbol**, donde **cada registro tiene un único padre y varios hijos**. Se usan para representar relaciones de dependencia rígidas.

Ejemplo:

Una empresa → departamentos → empleados.

✅ Ventajas:

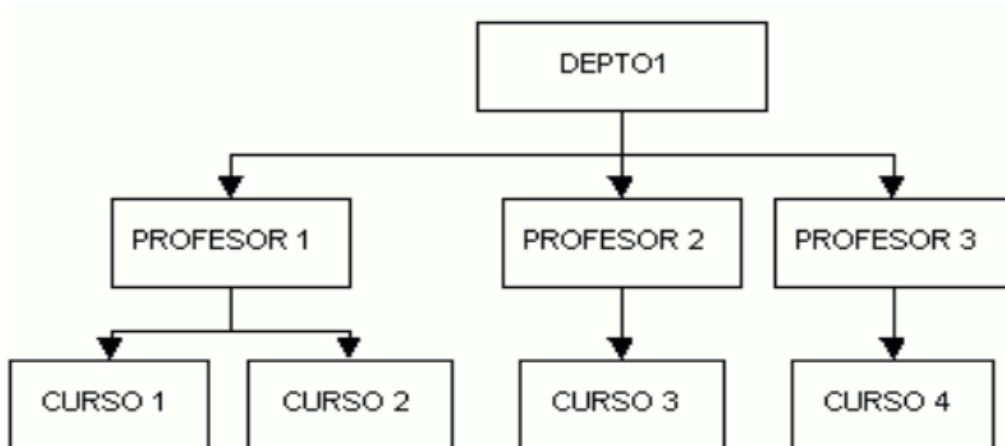
- Acceso rápido si la estructura es muy clara y predecible.

⚠️ Inconvenientes:

- Muy rígidas: difícil de modificar o adaptar.
- Obsoletas en muchos contextos.

📁 Ejemplo clásico:

- IBM IMS



4.5 Bases de Datos en Red

📌 ¿Qué son?

Parecidas a las jerárquicas, pero **más flexibles**. Permiten que un nodo tenga varios padres, representando relaciones **muchos a muchos**.

Ejemplo:

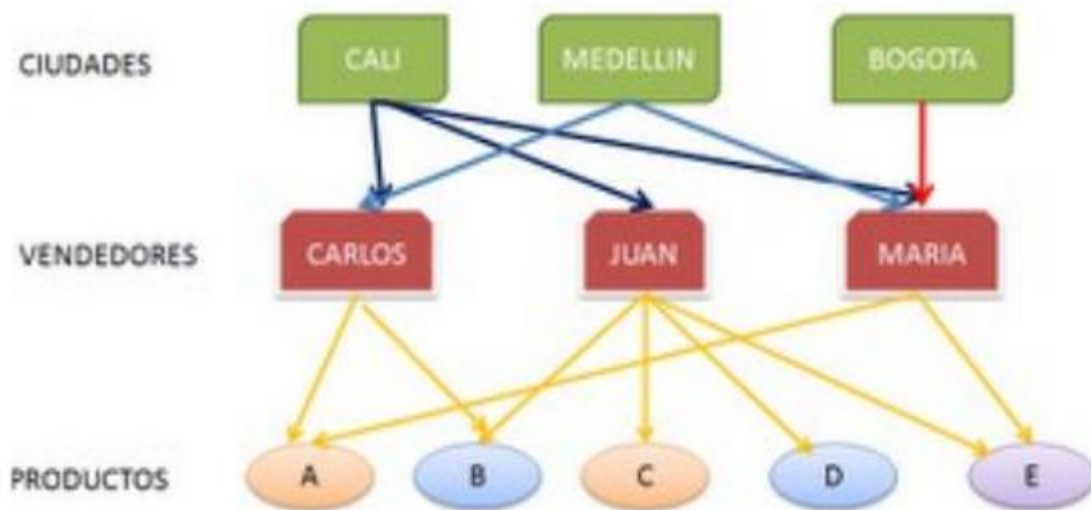
Un profesor puede dar varias asignaturas, y una asignatura puede ser impartida por varios profesores.

✓ Ventajas:

- Más versátiles que las jerárquicas.

⚠ Inconvenientes:

- Complejas de mantener.
- En desuso en la mayoría de los entornos actuales.



 Comparativa visual

Tipo	Estructura	Ideal para...	Ejemplos
Relacional (SQL)	Tablas	Sistemas empresariales clásicos	MySQL, PostgreSQL
NoSQL	Documentos, grafos	Big Data, apps web y móviles	MongoDB, Redis, Neo4j
Orientada a objetos	Objetos	Aplicaciones Java, C#	db4o, ObjectDB
Jerárquica	Árbol	Sistemas muy rígidos	IBM IMS
En red	Nodos interconectados	Modelos complejos	(en desuso)

🔗 Conclusión del apartado 4

Cada tipo de base de datos tiene ventajas e inconvenientes. No existe una “mejor” en general, sino que **dependen del contexto**:

- Para datos bien estructurados y relaciones claras, lo mejor son las bases de datos relacionales.
- Para datos muy dinámicos, en tiempo real o sin estructura fija, lo más adecuado puede ser NoSQL.
- **Para proyectos con programación orientada a objetos**, pueden aprovecharse bases de datos orientadas a objetos.

En este curso nos centraremos en el modelo **relacional (SQL)**, pero es importante conocer las demás opciones para entender su evolución y sus aplicaciones en el mundo real.

5. Conceptos básicos de estructura

Una base de datos relacional no solo almacena datos, sino que lo hace de forma **estructurada y lógica**, siguiendo un diseño que representa el mundo real.

Para comprender cómo se organiza la información dentro de una base de datos, es esencial conocer conceptos como:

- Entidades
- Atributos
- Tablas
- Registros
- Claves primarias
- Relaciones entre tablas

5.1 ¿Qué es una entidad?

Una **entidad** es un **objeto o concepto del mundo real** sobre el que se desea guardar información en la base de datos.

Ejemplos de entidades:

- Un cliente
- Un producto
- Un pedido
- Un profesor
- Una matrícula

Cada entidad se describe mediante una serie de **atributos** (características que la definen).

Por ejemplo, la entidad Cliente podría tener como atributos:

- Nombre
- Correo electrónico
- Dirección
- Teléfono

En la fase de diseño conceptual de una base de datos, utilizamos un **modelo entidad-relación (MER)**, donde representamos gráficamente cada entidad y sus atributos, así como las relaciones entre ellas.

5.2 De la entidad a la tabla: cómo se implementa

Cuando pasamos del modelo conceptual al modelo físico (es decir, cuando programamos la base de datos con SQL), **cada entidad se convierte en una tabla y cada atributo en una columna**.

Por tanto:

- Entidad → Tabla
- Atributo → Campo o columna
- Instancia de la entidad → Registro o fila

5.3 ¿Qué es una tabla?

Una **tabla** es la estructura básica de almacenamiento en una base de datos relacional. Es la **representación física** de una entidad.

- Cada **tabla** almacena información sobre un tipo concreto de objeto (una entidad).
- Las **columnas** son los atributos.
- Las **filas** son los registros (instancias de la entidad).



Ejemplo:

La tabla clientes representa la entidad Cliente.

id_cliente	nombre	email
1	Ana Pérez	ana@email.com
2	Luis Ramírez	luis@email.com

5.4 Campo o atributo

Un campo o atributo es cada una de las columnas de una tabla. Representa una característica específica de la entidad.

Ejemplo:

En la tabla productos, los atributos podrían ser:

- id_producto
- nombre
- precio
- stock

Cada campo se define con un **tipo de dato** (INT, VARCHAR, DATE, etc.) y puede tener reglas como NOT NULL, DEFAULT, UNIQUE, etc.

5.5 Registro o tupla

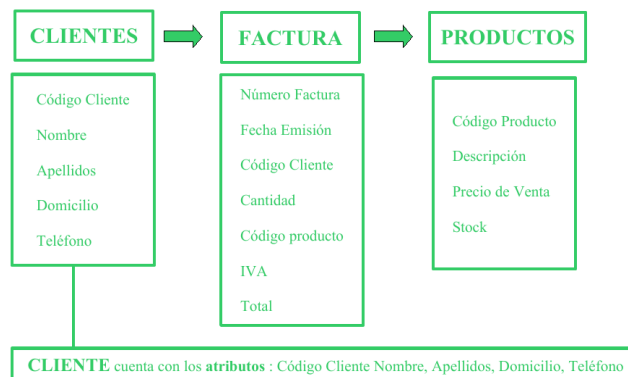
Un **registro** (también llamado **tupla**) es una **fila completa de una tabla**, es decir, una instancia concreta de la entidad.

Ejemplo:

Un registro de la tabla productos podría ser:

- id_producto: 3
- nombre: "Ratón inalámbrico"
- precio: 18.99
- stock: 25

Cada registro debe tener una **identificación única**, para poder distinguirlo del resto.



5.6 Clave primaria (Primary Key)

La **clave primaria** es uno de los elementos más importantes del diseño de una tabla.

Es un campo (o conjunto de campos) que **identifica de manera única** a cada registro de la tabla.

📌 Propiedades:

- No puede repetirse (**única**).
- No puede ser nula (**obligatoria**).
- Sirve de referencia para relacionar registros en otras tablas.

Ejemplo:

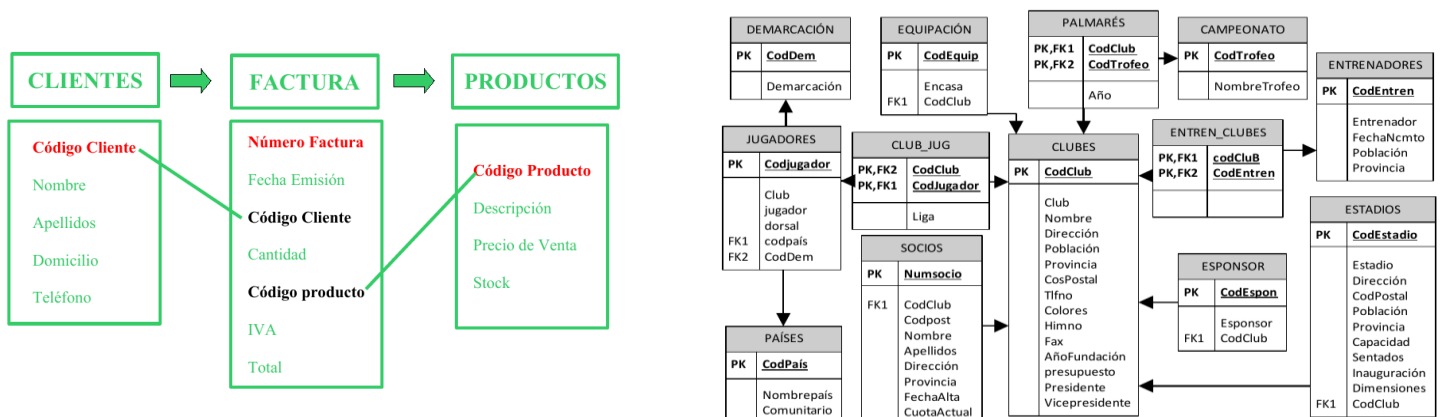
En la tabla clientes, la clave primaria puede ser id_cliente.

Sql -> id_cliente INT PRIMARY KEY

5.7 Relaciones entre tablas

Una base de datos relacional permite **enlazar información** entre distintas tablas sin duplicar datos. Estas conexiones se hacen mediante **relaciones**, que reflejan cómo se conectan las entidades entre sí.

Las relaciones se representan usando **claves externas (foreign keys)**, que hacen referencia a una clave primaria de otra tabla.



Ejemplo real:

La tabla pedidos puede tener un campo id_cliente que **apunta** al cliente que hizo el pedido.

```
SQL

CREATE TABLE pedidos (

id_pedido INT PRIMARY KEY,

id_cliente INT,

FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)

);
```

Tipos de relaciones más comunes:

Tipo	Ejemplo	Explicación
1 a 1	Persona ↔ DNI	Cada persona tiene un solo DNI
1 a muchos	Cliente → Pedidos	Un cliente puede tener varios pedidos
Muchos a muchos	Estudiantes ↔ Asignaturas	Un estudiante puede cursar muchas asignaturas y cada asignatura puede ser cursada por muchos estudiantes.

🔗 Conclusión del punto 4

- Cada entidad del mundo real (persona, producto, venta) se representa en la base de datos como una **tabla**.
- Las tablas se componen de atributos (columnas) y registros (filas).
- Las claves primarias y claves externas permiten relacionar la información entre tablas.
- Esta estructura modular permite construir sistemas complejos, sin redundancia y con gran flexibilidad.

Comprender la estructura interna es fundamental para diseñar bases de datos eficientes, normalizadas y preparadas para crecer.

Anexo: Objetos y Programación Orientada a Objetos (POO)

1. Introducción

La **Programación Orientada a Objetos (POO)** es uno de los paradigmas de programación más utilizados en el desarrollo de software moderno. Fue creada para **modelar el mundo real de forma más natural** en los programas informáticos.

En la POO, un programa se construye a partir de **objetos** que interactúan entre sí.

Este enfoque es fundamental para entender **las bases de datos orientadas a objetos**, que almacenan información directamente en forma de objetos, en lugar de utilizar las tradicionales tablas con filas y columnas.

2. ¿Qué es un objeto?

Un objeto es una estructura de datos que agrupa información (atributos) y funcionalidad (métodos).

Podemos imaginar un objeto como una “caja” que contiene datos y las operaciones que pueden realizarse sobre esos datos.

Ejemplo real:

Imagina un objeto Libro:

- **Atributos:** título, autor, número de páginas
- **Métodos:** abrir(), leer(), cerrar()

3. Clase y Objeto: la relación

☒ Clase

Es un **molde**, una **plantilla** abstracta que define cómo serán los objetos. Una clase describe:

- Qué atributos tendrán los objetos.
- Qué métodos podrán ejecutar.

☑ Objeto

Es una **instancia concreta** de una clase. Cada objeto es **único**, aunque comparta estructura con otros objetos de su misma clase.

🌐 Ejemplo:

python

```
class Persona:

    def __init__(self, nombre, edad):

        self.nombre = nombre

        self.edad = edad

juan = Persona("Juan", 25)
```

Aquí Persona es la clase y juan es un objeto de esa clase.

4. Componentes fundamentales de la POO

4.1 Atributos

Son las **propiedades** del objeto. Representan su **estado**.

Ejemplo: color, marca, precio de un coche.

4.2 Métodos

Son las funciones asociadas al objeto. Representan su comportamiento.

Ejemplo: encender(), acelerar(), frenar() en un coche.

4.3 Encapsulamiento

Es el principio de **ocultar los detalles internos** del objeto. Se accede a sus datos a través de métodos, no directamente.



4.4 Herencia

Permite que una clase nueva herede atributos y métodos de otra clase.

Ejemplo: una clase Empleado puede heredar de Persona.

4.5 Polimorfismo

Permite que varios objetos distintos respondan a un mismo método de forma diferente.

Ejemplo: si tenemos objetos Coche y Avión, ambos pueden tener el método arrancar(), pero su funcionamiento será distinto en cada uno.

5. Ventajas de la programación orientada a objetos

Ventaja	Explicación
Modularidad	El código se divide en piezas independientes (objetos), más fáciles de mantener
Reutilización de código	Gracias a la herencia y las clases reutilizables
Mantenimiento más sencillo	Los cambios en una parte del sistema no afectan a las demás
Representación natural	Se modela el mundo real con objetos que tienen atributos y comportamientos
Mejor organización del código	Se favorece la estructura lógica y la escalabilidad del proyecto

6. ¿Qué relación tiene esto con las bases de datos?

En las bases de datos **relacionales** (SQL), tenemos que convertir los objetos en **tablas** y **columnas**: a esto se le llama **mapeo objeto-relacional (ORM)**. Pero esta conversión puede ser compleja o costosa en tiempo.

En cambio, las bases de datos orientadas a objetos permiten almacenar los objetos directamente, sin necesidad de convertirlos en otro formato.



☞ Un objeto de la aplicación es el mismo que se almacena en la base de datos.

Esto facilita mucho el trabajo cuando:

- La aplicación está escrita en un lenguaje orientado a objetos.
- Los datos tienen estructuras complejas y relaciones jerárquicas.

7. Ejemplo comparativo

Supón que en una aplicación tienes esta clase en Python:

python

```
class Alumno:

    def __init__(self, nombre, edad, curso):

        self.nombre = nombre

        self.edad = edad

        self.curso = curso
```

En una base de datos relacional (SQL) se almacenaría así:

nombre	edad	curso
"María"	18	"1º DAM"

Pero si usamos una **base de datos orientada a objetos**, se podría guardar directamente como:

Objeto Alumno:

- nombre = "María"
- edad = 18
- curso = "1º DAM"

8. Conclusión

- La **POO** es una forma de pensar y programar basada en objetos que simulan cosas reales.
- Una **base de datos orientada a objetos** permite almacenar directamente esos objetos, lo que **evita duplicidad de estructuras y mejora el rendimiento** de ciertas aplicaciones.
- Aunque en este curso trabajaremos principalmente con bases de datos relacionales, es fundamental comprender este paradigma para poder **valorar cuándo usar uno u otro enfoque**.

Glosario (Orden Alfabético) – Sesión 1

Atributo:

Característica de una entidad. Se corresponde con una columna en una tabla.

Back-up:

Copia de seguridad de los datos para poder restaurarlos en caso de pérdida o fallo.

Base de datos (BD):

Colección organizada de datos accesibles electrónicamente para su gestión y consulta.

Campo:

Columna específica de una tabla, que contiene un tipo determinado de dato.

Clase:

Plantilla o molde para crear objetos en POO. Define qué datos y comportamientos tendrán los objetos.

Clave externa:

Campo que conecta una tabla con otra, estableciendo una relación entre ambas.

Clave primaria:

Campo que identifica de forma única cada registro dentro de una tabla.

Concurrencia:

Capacidad de una base de datos de permitir el acceso simultáneo de varios usuarios sin errores.

Consulta:

Instrucción SQL que permite recuperar datos específicos de una base de datos.

Dato:

Unidad mínima de información sin contexto; puede ser un número, palabra o símbolo.

Encapsulamiento:

Principio de ocultar los detalles internos de un objeto y acceder a ellos mediante métodos definidos.

Entidad:

Objeto del mundo real que se representa en una base de datos, como un cliente, producto o empleado.

Herencia:

Propiedad de una clase que permite heredar atributos y métodos de otra clase.

Información:

Datos organizados y contextualizados que tienen significado y utilidad.

Instancia:

Objeto concreto creado a partir de una clase.

Integridad referencial:

Regla que garantiza que las claves externas correspondan a registros existentes.

Interfaz gráfica (GUI):

Aplicación visual que facilita la gestión de una base de datos sin usar código directamente.



Modelo entidad-relación (MER):

Representación gráfica del diseño de una base de datos antes de implementarla en SQL.

Modelo relacional:

Forma de organizar los datos en tablas con relaciones definidas entre ellas.

Método:

Función definida dentro de una clase que determina el comportamiento de los objetos.

Normalización:

Proceso de organizar los datos para reducir redundancia y mejorar integridad.

NoSQL:

Bases de datos no relacionales que almacenan datos en estructuras como documentos, pares clave-valor o grafos.

Objeto:

Estructura que representa una entidad en POO, incluyendo sus atributos y métodos.

POO (Programación Orientada a Objetos):

Paradigma de programación basado en objetos que combinan datos y funciones.

Registro / Tupla:

Fila de una tabla que representa una instancia completa de una entidad.

Relación:

Vínculo lógico entre tablas de una base de datos basado en claves.

SGBD (DBMS):

Sistema Gestor de Base de Datos. Software que permite gestionar, consultar y proteger los datos.

SQL:

Lenguaje de consulta estructurado utilizado para interactuar con bases de datos relacionales.

Tabla:

Estructura que almacena los datos en filas (registros) y columnas (atributos).

Tipo de dato:

Clasificación que determina el tipo de contenido permitido en una columna: texto, número, fecha, etc.

Transacción:

Conjunto de operaciones que se ejecutan como una unidad indivisible en una base de datos.